

1. Signal Agent — *LLM, but mostly glue*

- SerpAPI: `google_news` search for a live disruption (e.g. "Shanghai port typhoon closure"). Extract headline + URL.
- Fallback chain (demo insurance): live result → `cache/serp_cache.json` → hardcoded scenario block from `erp_bundle["scenario"]`. The demo never dies on a network call.
- Match vessel name → PO-10291 → revised ETA from scenario. LLM used only to parse news text into the `disruption` dict (JSON mode).
- Trace log entry: *"Signal Agent: detected Typhoon Mitag via [source], matched to PO-10291, ETA revised +12 days."*

2. Impact Engine — *pure Python, zero LLM. This is your correctness moat.*

- Input: `disruption` + `erp_bundle.json`
- Compute: $\text{consumption} = \text{MPS} \times \text{BOM qty}$ ($40 \times 2 = 80/\text{day}$) → $\text{TTS} = 400/80 = 5\text{d}$ → stockout Oct 7 → $\text{TTR} = 16\text{d}$ → shortage window Oct 7–18
- Traverse BOM: STCOIL → Motor Assembly → WO-7781/7782/7783 → SO-55102 ACME → \$120k OTIF exposure
- Output: `impact` dict + `bom_trace` for the UI tree
- Testable ground truth — Person B writes `test_engines.py` with these exact numbers. Judges ask "how do you know it's right?" → you show the test suite.

3. Recovery Agent — *LLM, the creative one*

- Prompt gives it: `impact` + supplier list (WITHOUT hinting at their constraints — this is critical; the vetoes must be genuine, not scripted) + instruction to propose exactly 4 options, one per category: expedited/alternate supply, alternate supplier, production reschedule, material substitution.
- JSON mode, temperature ~0.3, required fields: `supplier`, `qty`, `arrival_date`, `est_cost_usd`.
- The LLM will naturally generate options that hit C1/C3/C4/C6 — your dataset was built so they do.

4. Constraint Engine — *pure Python, your headline feature*

- Implements C1–C8 from the rulebook as functions. Each returns `{status, rules_triggered[], reason}`.
- Air freight survivor gets flagged: `rules_triggered: ["C5"]`, status feasible but `requires_vp_approval: true`.

- Veto messages must be human-readable: *"VETOED by C1 (Supplier lead time): arrival 2026-11-01 exceeds revised ETA 2026-10-18 — cannot bridge the shortage window."*

5. Human Gate — *LangGraph* `interrupt_before`

- Graph pauses. Streamlit renders survivors with the costed trade-off and an Approve button.
- Button click → `graph.invoke(None, config=thread)` resumes with `decision` set.
- Reject → graph ends gracefully with "no feasible option; escalation to S&OE board" (shows you handle dead ends).

6. Audit + Summary Agent — *LLM, the polish*

- Appends to `audit/audit.jsonl`: full decision trail (options considered, vetoes with rule IDs, approver, timestamps, cost delta).
- LLM writes a 3-sentence exec summary for the UI/judges.